Hit Forensics.

What makes a song a hit?

TEAM

Kevin Li · Tommy Ou · Ronnie Wang · Kevin Xue

COURSE

CIS 5500
Spring 2026

INSTITUTION

University of
Pennsylvania

The music industry runs on folklore. We built a queryable forensic system.

Spotify knows what songs sound like. Billboard knows what songs sold. Joining them lets you ask questions no existing tool answers — at the level of a single track, an artist, a decade, or a collaboration.

Two datasets. One join.

SPOTIFY · AUDIO

What it sounds like

Per-track audio fingerprint — 11 raw features (danceability, energy, valence, tempo, acousticness, instrumentalness, speechiness, liveness, loudness, key, mode). The five most discriminative are stored as a cube column for k-NN.

587,000 raw → **447,324**

Tracks
447,324 rows

Artists
79,822 rows

Source
Kaggle · Spotify API

Genres
125 distinct

BILLBOARD · CHARTS

What it sold

Weekly Hot 100 entries — current rank, peak rank, weeks-on-chart — over 65 years of American radio and retail.

~330,000 raw → **230,351**

Span
1958 — 2021

Granularity
One row · track · week

Source
Kaggle · Hot 100 archive

Coverage
65 years

THE BRIDGE · TRACK_ARTISTS JUNCTION

Resolves the many-to-many between tracks and artists, with `is_primary` flagging the lead. **569,749** ROWS

From two CSVs to one canonical track per row.

STEP 01 · NORMALIZE

Text normalization

NFKD unicode decomposition strips accents; everything lowercased; punctuation removed before any matching happens.

```
"Beyoncé — Déjà Vu" → "beyonce deja vu"
```

STEP 02 · TOKENIZE

Strip collaboration markers

"featuring", "ft", "with", "&" all collapse into a clean artist token set, so a Billboard credit can match a Spotify primary cleanly.

```
"Drake feat. Rihanna" → {drake, rihanna}
```

STEP 03 · DEDUPLICATE

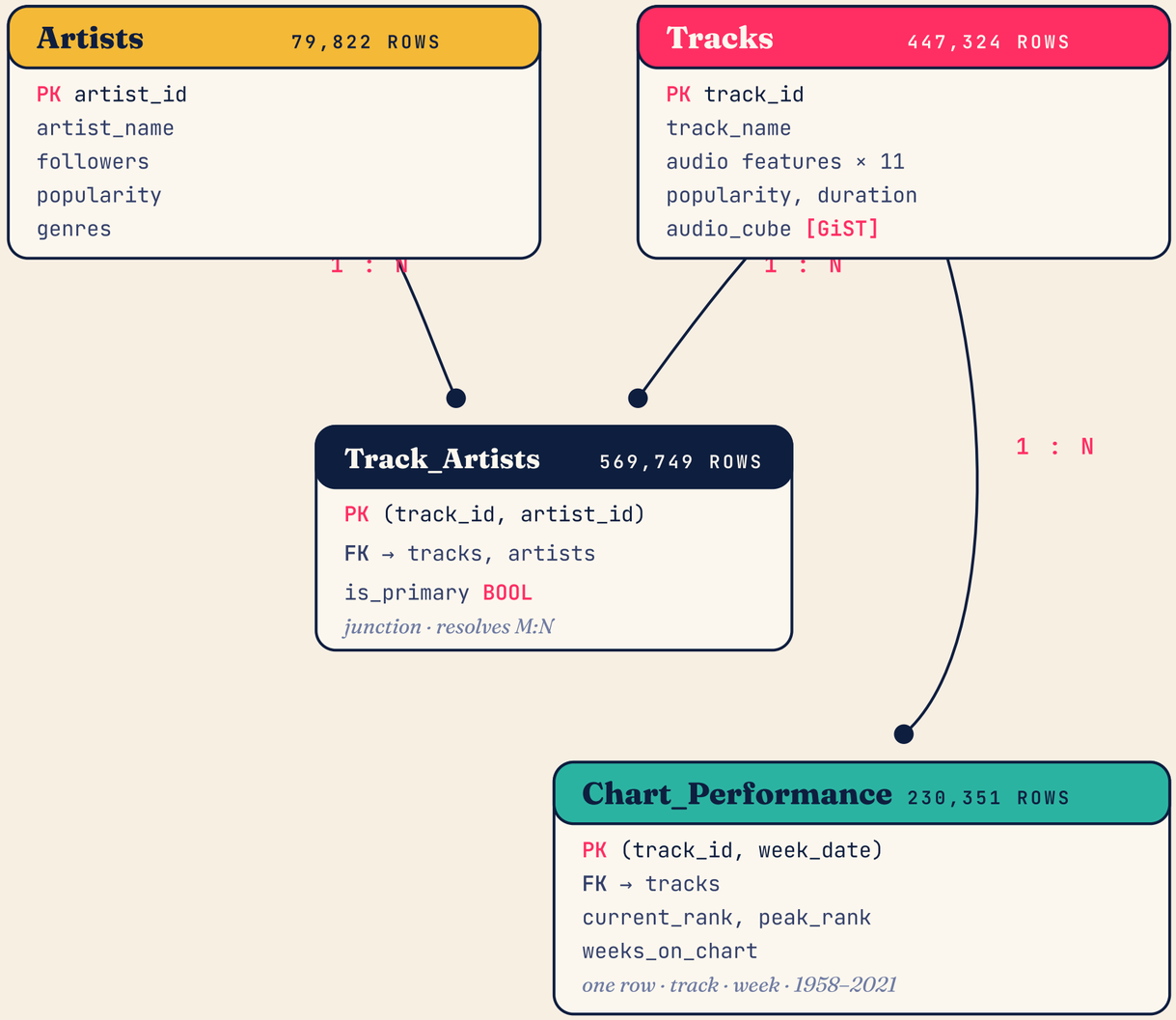
Sort by popularity, strict 1:1

Bi-directional subset match on artist tokens; sort matches descending by Spotify popularity; then strict dedup on `(date, song, artist)`.

```
canonical = max(popularity)
```

CARTESIAN EXPLOSION · SOLVED

Loose subset matching let one Billboard row match the original, the acoustic, the radio edit, and the remix. Sorting by popularity before strict dedup forces a 1:1 mapping to the canonical Spotify version.



3NF · JUSTIFIED

No partial, no transitive dependencies.

Tracks — TRACK_ID

Every audio feature depends only on track_id.

Artists — ARTIST_ID

Name, followers, popularity depend only on artist_id.

Track_Artists — (TRACK_ID, ARTIST_ID)

is_primary depends on both keys together — minimal composite key.

Chart_Performance — (TRACK_ID, WEEK_DATE)

Rank metrics depend on the full composite key.

Four layers. Database does the work.

DATABASE

PostgreSQL 16 *on AWS RDS*

cube + GiST
pg_trgm GIN
materialized view

↓

BACKEND

Node.js *+ Express*

three route files
pg connection pool
Vitest + Supertest

↓

FRONTEND

React + Vite *· Recharts*

React Router 6
9 pages
client-side routing

↓

PIPELINE

Python + pandas *· offline ETL*

entity resolution
dedup + cleaning
writes to RDS

Nine pages. Each backed by one production query.

Q1 · FUZZY

Search

Trigram-indexed lookup over 447k track names.

Q2 · Q4 · Q6

Track Detail

Song Spotlight

Audio fingerprint radar, weekly chart trajectory, k-NN audio twins.

Q3

Artist Detail

Full discography with best peak and total weeks per track.

Q5

Top Charts

Longest-charting tracks, all eras combined.

Q7

Audio Workbench

Filter by feature ranges; rank by per-decade longevity percentile.

Q8

Trajectory Gallery

Five archetypes — Flash, Sleeper, Slow Burn, Sustained, Other.

Q9

Era Decoder

Top-10 hits ≥ 2σ from their decade's audio mean.

Q10

Collab Chemistry

Joint-vs-solo uplift across every charting artist pair.

–

Home

Editorial entry point with featured queries and project lede.

Q6

AUDIO TWIN FINDER

k-NN over a 5-D audio cube.

For any track, return the ten others with the most similar audio fingerprint — across the five most discriminative Spotify features.

- **cube** extension casts five audio features to `audio_cube`.
- The `<->` operator computes Euclidean distance directly on the cube.
- **GiST index** on `audio_cube` turns a 500k-row sequential scan into a sub-second k-NN traversal.

21.8× 2,166 MS → 99 MS

```sql
-- Q6 · Audio Twin Finder · k-NN via cube + GiST
SELECT t.track_id,
       t.track_name,
       ROUND((t.audio_cube <-> ref.cube)::numeric, 4)
           AS audio_distance
  FROM tracks t
  CROSS JOIN (
     SELECT audio_cube AS cube
       FROM tracks
      WHERE track_id = $1
  ) ref
 WHERE t.track_id <> $1
   AND t.audio_cube IS NOT NULL
 ORDER BY t.audio_cube <-> ref.cube
 LIMIT 10;
```

Q8

TRAJECTORY ARCHETYPES

Window functions classify chart shape.

Every charting track has a shape: how fast it peaked, how long it stayed. We bucket the 230k chart-week rows into five archetypes.

- Multi-CTE pipeline: aggregate, then compute `FIRST_VALUE` debut and peak metrics per track.
- **CASE classifier** partitions tracks: Flash, Sleeper, Slow Burn, Sustained Hit, Other.
- Final join back to `tracks` reports the average audio profile per archetype.

5 ARCHETYPES OVER 65 YEARS

```sql
-- Q8 · Trajectory Archetypes · CASE-based classifier (CASE truncated)
WITH debut_peak AS (
  SELECT DISTINCT track_id,
    FIRST_VALUE(current_rank) OVER (
      PARTITION BY track_id ORDER BY week_date) AS debut_rank,
    FIRST_VALUE(week_date)    OVER (
      PARTITION BY track_id ORDER BY current_rank, week_date) AS peak_week
    FROM chart_performance
),
labeled AS (
  SELECT cs.track_id,
    CASE
      WHEN total_weeks ⩽ 10 AND peak_week - debut_date ⩽ 21  THEN 'Flash'
      WHEN debut_rank > 50 AND peak_week - debut_date ⩾ 140 THEN 'Sleeper'
      WHEN peak_week - debut_date ⩾ 70 AND total_weeks ⩾ 25 THEN 'Slow Burn'
      WHEN best_rank ⩽ 10 AND total_weeks ⩾ 15           THEN 'Sustained Hit'
      ELSE 'Other'
    END AS archetype
    FROM chart_stats cs JOIN debut_peak dp USING (track_id)
)
SELECT archetype, COUNT(*), AVG(danceability), AVG(energy), AVG(valence)
  FROM labeled JOIN tracks USING (track_id)
 GROUP BY archetype ORDER BY COUNT(*) DESC;
```

Pre & post optimization.

Q	DESCRIPTION	PRE (MS)	POST (MS)	SPEEDUP
Q6	**Audio Twin Finder** K-NN	2,166.792	99.135	~21.8×
Q7	**Audio Workbench**	742.410	58.447	~12.7×
Q8	**Trajectory Archetypes**	2,387.906	1,964.763	~1.2×
Q9	**Era Decoder** Z-SCORE	1,415.652	45.819	~30.9×
Q10	**Collab Chemistry**	1,131.467	483.422	~2.3×

METHOD *EXPLAIN ANALYZE on the live RDS instance, hot cache, median of three runs. Q6 and Q9 are the headline wins — both replace inline aggregation with a precomputed structure.*

Three techniques. One result each.

01

MATERIALIZED VIEW

mv_charted_tracks

Precomputes per-track *best_peak*, *total_weeks*, *debut_decade*. Shifts aggregation from read-time to write-time.

Applied to · Q7 · Q9 · Q10

Q9 · 1,415 ms → **45.8 ms**

02

CUBE + GIST

11-D audio_cube

Replaces a 500k-row sequential scan and millions of POWER / SQRT calls with a tree-indexed Euclidean operator.

Applied to · Q6

Q6 · 2,166 ms → **99.1 ms**

03

B-TREE COMPOSITE

chart_performance (track_id, week_date)

Gives the planner pre-sorted data for FIRST_VALUE and NTILE window functions; junction lookups stay O(log n).

Applied to · Q4 · Q8 · Q10

Q8 · 2,388 ms → **1,965 ms**

The 1.2× understates it. B-tree's real win is qualitative — without (track_id, week_date) *pre-sorted, the planner falls back to sort-then-scan and* FIRST_VALUE / NTILE *window functions never shape into a clean plan.*

CACHING ***The materialized view is our cache.*** *No server-side query result caching — the data is largely static, and shifting aggregation to write-time gets us the same hit ratio without invalidation logic.* `pg_trgm` *GIN index on track and artist names accelerates Q1's fuzzy search.*

Three problems. Three rewrites.

01 Cartesian explosion	One Billboard row matched the original, the acoustic, the radio edit, and the remix. **Fix:** in the Python pipeline, sort matches descending by Spotify popularity, then strict dedup on `(date, song, artist)` for a 1:1 mapping to the canonical version.
02 Q10 nested-loop bottleneck	`NOT EXISTS` against the 569k-row junction triggered a 5-minute timeout. **Fix:** a CTE with `GROUP BY track_id HAVING COUNT(*) = 1` partitions solo tracks in a single pass. **5 min → 1.2 s**, no schema change.
03 Redundant aggregations	Era Decoder and Workbench were recomputing decade stats and best_peak / total_weeks *per request*. **Fix:** the `mv_charted_tracks` materialized view; routes point at the view instead of running the inline CTE.

LIVE · LOCALHOST:5173

Demo.

Switching to the browser — Song Spotlight first, then the Workbench.

QUESTIONS WELCOME

Thank you.

TEAM

Kevin Li · Tommy Ou · Ronnie Wang · Kevin Xue

COURSE

CIS 5500
Spring 2026

Q&A

Ask us about the Cartesian explosion.